XML 데이터 파싱 및 데이터베이스 구축 프로젝트 결과

데이터베이스시스템 학기 프로젝트

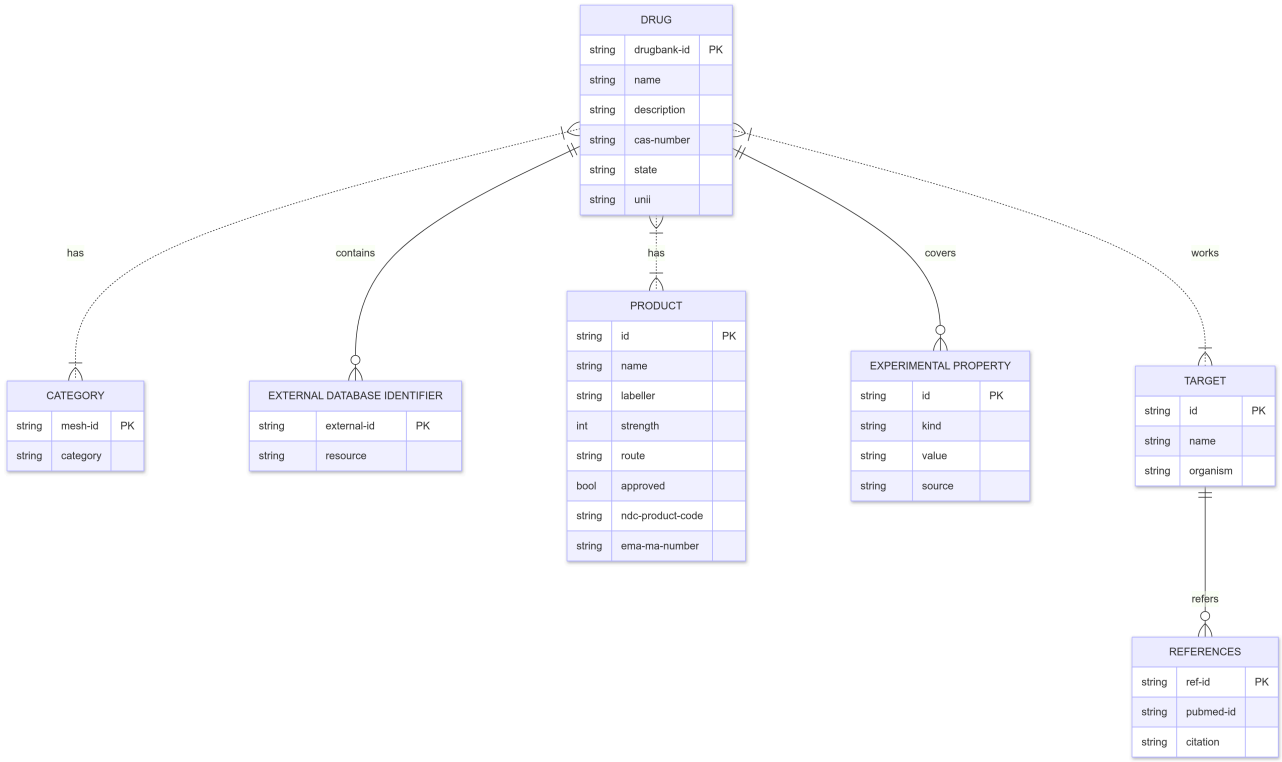
박종현, 공과대학 컴퓨터정보통신공학과
jonghyeon@jnu.ac.kr

최종 과제 목표

1. ERD, relational schema 작성하여 제출
2. Database의 구축된 각 테이블에 description 캡쳐하여 제출
3. Database 및 테이블 생성 코드 제출
4. 각 테이블에 index 설정 코드 및 설정결과 제출
5. 각 테이블 별로 포함된 tuple 예시 출력하여 제출

1. ERD, Relational Schema

1.1. ERD ——



1.2. Relational Schema ——

약물 `drugs`

약물 ID `id`	약물 이름 `name`	설명 `description`	CAS 번호 `cas_number`	상태 `state`	UNII `unii`
DB00039	Palifermin	Palifermin is a recombinant human keratinocyte growth factor (KGF). It is 140 residues long, and is produced using E. coli.	162394-19-6	liquid	QMS40680K6

		Palifermin was granted FDA approval on 15 December 2004.[L17933]			
DB00040	Glucagon	Glucagon is a 29 amino acid hormone used as a diagnostic aid in radiologic exams to temporarily inhibit the movement of the gastrointestinal tract and to treat severe hypoglycemia. [L7634,L7637,L7640,L7643,L8519] Glucagon raises blood sugar through activation of hepatic glucagon receptors, stimulating glycogenolysis and the release of glucose. [L7640,L7643] Glucagon was granted FDA approval on 14 November 1960.[L7631]	16941-32-5	liquid	76LA80IG2G
DB00041	Aldesleukin	Aldesleukin, a lymphokine, is produced by recombinant DNA technology using a genetically engineered E. coli strain containing an analog of the human interleukin-2 gene. Genetic engineering techniques were used to modify the human IL-2 gene, and the resulting expression clone encodes a modified human interleukin-2. This recombinant form differs from native interleukin-2 in the following ways: a) Aldesleukin is not glycosylated because it is derived from E. coli; b) the molecule has no N-terminal alanine; the codon for this amino acid was deleted during the genetic engineering procedure; c) the molecule has serine substituted for cysteine at amino acid position 125.	110942-02-4	liquid	M89N0Q7EQR

카테고리 categories

카테고리 ID id	카테고리 이름 name
D058917	Adenosine A2 Receptor Antagonists
D058919	Purinergic P2 Receptor Antagonists
D058921	Purinergic P2Y Receptor Antagonists
D058924	Thienopyridines
D058950	Melanoma-Specific Antigens
D058955	Metalloids

약물-카테고리 관계 drug_categories

약물 ID drug_id	카테고리 ID category_id
DB16267	D065693
DB00030	D065694
DB00046	D065694
DB00047	D065694

외부 DB 식별자 external_db_identifiers

ID id	외부 ID external_id	유래 약물 derived_from	리소스 resource
11f03d37-7e2e-82d8-8f74-e2c01f351b76	PA164760860	DB00059	PharmGKB
11f03d37-7e2e-8512-8f74-e2c01f351b76	Pegaspargase	DB00059	Wikipedia
11f03d37-7e2e-8742-8f74-e2c01f351b76	CHEMBL2108546	DB00059	ChEMBL
11f03d37-7e2e-8940-8f74-e2c01f351b76	34132	DB00059	RxCUI

상품 products

ID id	유래 약물 derived_from	상품명 name	제조사 labeller	강도 strength	투여 경로 route	승인 여부 approved	NDC 제품 코드 ndc_product_code
11f03d37-815f-8ef2-8f74-e2c01f351b76	DB00008	Pegasys	Genentech, Inc.	180 ug/0.5mL	Subcutaneous	1	0004-0365
11f03d37-815f-9118-8f74-e2c01f351b76	DB00008	Pegasys	Hoffmann La Roche	180 mcg / 0.5 mL	Subcutaneous	1	
11f03d37-815f-9366-8f74-e2c01f351b76	DB00008	Pegasys	Hoffmann La Roche	180 mcg / mL	Subcutaneous	1	
11f03d37-815f-956e-8f74-e2c01f351b76	DB00008	Pegasys	Zr Pharma& Gmbh	180 ug/1mL	Subcutaneous	1	82154-0449
11f03d37-815f-979e-8f74-e2c01f351b76	DB00008	Pegasys	Zr Pharma& Gmbh	180 ug/0.5mL	Subcutaneous	1	82154-0451
11f03d37-815f-99a6-8f74-e2c01f351b76	DB00008	Pegasys	Zr Pharma& Gmb H	135 µg	Subcutaneous	1	
11f03d37-815f-a284-8f74-e2c01f351b76	DB00008	Pegasys	Zr Pharma& Gmb H	135 µg	Subcutaneous	1	

실험 속성 experimental_properties

ID id	유래 약물 derived_from	종류 kind	값 value	단위 unit
11f03d37-8e88-0488-8f74-e2c01f351b76	DB04825	Melting Point	36.5 °C	PhysProp
11f03d37-8e88-06b8-8f74-e2c01f351b76	DB04826	Melting Point	96 °C	PhysProp
11f03d37-8e88-08fc-8f74-e2c01f351b76	DB04826	Boiling Point	159 °C at 2.00E-02 mm Hg	PhysProp
11f03d37-8e88-0b9a-8f74-e2c01f351b76	DB04827	Water Solubility	4.8E+005 mg/L (at 15 °C)	SEIDELL,A (1941)
11f03d37-8e88-0eb0-8f74-e2c01f351b76	DB04827	Melting Point	49 °C	PhysProp
11f03d37-8e88-1162-8f74-e2c01f351b76	DB04827	Boiling Point	185 °C	PhysProp

작용 대상 targets

대상 ID id	대상 이름 name	생물체 organism
BE0000140	Solute carrier family 12 member 6	Humans
BE0000141	Sodium channel protein type 1 subunit alpha	Humans
BE0000142	Penicillin-binding protein 2B	Streptococcus pneumoniae serotype 4 (strain ATCC BAA-334 / TIGR4)
BE0000143	Tubulin beta chain	Humans
BE0000144	Retinoic acid receptor gamma	Humans
BE0000145	Dopamine D5 receptor	Humans

약물-작용 대상 관계 drug_targets

약물 ID drug_id	대상 ID target_id
DB12010	BE0009442
DB12010	BE0009443
DB12010	BE0009444
DB12010	BE0009445
DB12010	BE0009446
DB12010	BE0009447
DB12010	BE0009448
DB12010	BE0009449

참조 문헌 references

참조 ID ref_id	PubMed ID pubmed_id	인용 citation

A10031	2550053	Rudy B, Dubois H, Mink R, Trommer WE, McIntyre JO, Fleischer S: Coenzyme binding by 3-hydroxybutyrate dehydrogenase, a lipid-requiring enzyme: lecithin acts as an allosteric modulator to enhance the affinity for coenzyme. Biochemistry. 1989 Jun 27;28(13):5354-66.
A10032	38961	Smith CM, Plaut GW: Activities of NAD-specific and NADP-specific isocitrate dehydrogenases in rat-liver mitochondria. Studies with D-threo-alpha-methylisocitrate. Eur J Biochem. 1979 Jun;97(1):283-95.
A10033	16983536	Drew DP, Lunde C, Lahnstein J, Fincher GB: Heterologous expression of cDNAs encoding monodehydroascorbate reductases from the moss, Physcomitrella patens and characterization of the expressed enzymes. Planta. 2007 Mar;225(4):945-54.
A10034	237755	Inano H, Tamaoki B: Relationship between steroids and pyridine nucleotides in the oxido-reduction catalyzed by the 17 beta-hydroxysteroid dehydrogenase purified from the porcine testicular microsomal fraction. Eur J Biochem. 1975 May 6;53(2):319-26.
A10035	15755483	Huber R, Riepe MW: Improved posthypoxic recovery in vitro on treatment with drugs used for secondary stroke prevention. Neuropharmacology. 2005 Mar;48(4):558-65.
A10036	15152097	Hedl M, Rodwell VW: Inhibition of the class II HMG-CoA reductase of Pseudomonas mevalonii. Protein Sci. 2004 Jun;13(6):1693-7.
A10037	14511985	Li S, Wagner CA, Friesen JA, Borst DW: 3-hydroxy-3-methylglutaryl-coenzyme A reductase in the lobster mandibular organ: regulation by the eyestalk. Gen Comp Endocrinol. 2003 Nov;134(2):147-55.
A10038	17197218	McKinlay JB, Shachar-Hill Y, Zeikus JG, Vieille C: Determining Actinobacillus succinogenes metabolic pathways and fluxes by NMR and GC-MS analyses of 13C-labeled metabolic product isotopomers. Metab Eng. 2007 Mar;9(2):177-92. Epub 2006 Nov 17.
A10039	2692566	Grant GA: A new family of 2-hydroxyacid dehydrogenases. Biochem Biophys Res Commun. 1989 Dec 29;165(3):1371-4.

약물-참조 문헌 관계 `drug_references`

약물 ID `drug_id`	참조 ID `ref_id`
DB00157	A10001
DB00157	A10002
DB00157	A10003
DB00157	A10004
DB00162	A10004

DB00157	A10005
DB00126	A100057
DB00157	A10006
DB00157	A10007
DB00157	A10008
DB00157	A10009

2. 구축된 각 테이블의 DESCRIPTION

2-desc-tables.sql - 2-desc-tables.sql

Result 1-3

2 rows

	Field	Type	Null	Key	Default	Extra
1	id	varchar(20)	NO	PRI	<null>	
2	name	varchar(256)	NO		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-4

2 rows

	Field	Type	Null	Key	Default	Extra
1	drug_id	varchar(20)	NO	PRI	<null>	
2	category_id	varchar(20)	NO	PRI	<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-11

2 rows

	Field	Type	Null	Key	Default	Extra
1	drug_id	varchar(20)	NO	PRI	<null>	
2	ref_id	varchar(20)	NO	PRI	<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-9

2 rows

	Field	Type	Null	Key	Default	Extra
1	drug_id	varchar(20)	NO	PRI	<null>	
2	target_id	varchar(20)	NO	PRI	<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-2

6 rows

	Field	Type	Null	Key	Default	Extra
1	id	varchar(20)	NO	PRI	<null>	
2	name	varchar(256)	NO		<null>	
3	description	text	YES		<null>	
4	cas_number	varchar(50)	YES		<null>	
5	state	varchar(50)	YES		<null>	
6	unii	varchar(50)	YES		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-7

5 rows

	Field	Type	Null	Key	Default	Extra
1	id	binary(16)	NO	PRI	<null>	
2	derived_from	varchar(20)	NO	MUL	<null>	
3	kind	varchar(50)	NO		<null>	
4	value	varchar(256)	NO		<null>	
5	unit	varchar(1000)	YES		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-5

4 rows

	Field	Type	Null	Key	Default	Extra
1	id	binary(16)	NO	PRI	<null>	
2	external_id	varchar(512)	NO		<null>	
3	derived_from	varchar(20)	NO	MUL	<null>	
4	resource	varchar(50)	NO		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-6

9 rows

	Field	Type	Null	Key	Default	Extra
1	id	binary(16)	NO	PRI	<null>	
2	derived_from	varchar(20)	NO	MUL	<null>	
3	name	varchar(512)	NO		<null>	
4	labeller	varchar(256)	YES		<null>	
5	strength	varchar(50)	YES		<null>	
6	route	text	YES		<null>	
7	approved	tinyint(1)	YES		0	
8	ndc_product_code	varchar(20)	YES		<null>	
9	ema_ma_number	varchar(20)	YES		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 1-10

3 rows

	Field	Type	Null	Key	Default	Extra
1	ref_id	varchar(20)	NO	PRI	<null>	
2	pubmed_id	varchar(20)	NO		<null>	
3	citation	text	NO		<null>	

2-desc-tables.sql - 2-desc-tables.sql

Result 2-8

3 rows

	Field	Type	Null	Key	Default	Extra
1	id	varchar(20)	NO	PRI	<null>	
2	name	varchar(256)	NO		<null>	
3	organism	varchar(256)	YES		<null>	

3. Database 및 테이블 생성 코드

3-init-database.sql

```sql
DROP DATABASE `drugbank_db`

CREATE DATABASE IF NOT EXISTS `drugbank_db`
DEFAULT CHARACTER SET utf8mb4
DEFAULT COLLATE utf8mb4_unicode_ci;

USE `drugbank_db`;

CREATE TABLE IF NOT EXISTS `drugs` (
  `id` VARCHAR(20) NOT NULL PRIMARY KEY,
  `name` VARCHAR(256) NOT NULL,
  `description` TEXT,
  `cas_number` VARCHAR(50),
  `state` VARCHAR(50),
  `unii` VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS `categories` (
  `id` VARCHAR(20) NOT NULL PRIMARY KEY,
  `name` VARCHAR(256) NOT NULL
);

-- Connections between drugs and categories
CREATE TABLE IF NOT EXISTS `drug_categories` (
  `drug_id` VARCHAR(20) NOT NULL,
  `category_id` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`drug_id`, `category_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`category_id`) REFERENCES `categories`(`id`) ON DELETE CASCADE
```

```sql
);

CREATE TABLE IF NOT EXISTS `external_db_identifiers` (
  -- id: Specially converted from uuid
  `id` BINARY(16) NOT NULL PRIMARY KEY,
  -- `external_id` can be duplicated across different resources.
  -- so `id` is denoted as a primary key to identify the record uniquely.
  `external_id` VARCHAR(512) NOT NULL,
  `derived_from` VARCHAR(20) NOT NULL,
  `resource` VARCHAR(50) NOT NULL,
  FOREIGN KEY (`derived_from`) REFERENCES `drugs`(`id`) ON DELETE CASCADE
);

-- Connections between drugs and external databases
/*
-- No more required: `dervied_from` in `external_db_identifiers` table references
`drug_id` in `darugs` table.
CREATE TABLE IF NOT EXISTS `drug_external_ids` (
  `drug_id` VARCHAR(20) NOT NULL,
  `external_id` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`drug_id`, `external_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`external_id`) REFERENCES `external_db_identifiers`(`external_id`) ON
DELETE CASCADE
);
*/

CREATE TABLE IF NOT EXISTS `products` (
  `id` BINARY(16) NOT NULL PRIMARY KEY,
  `derived_from` VARCHAR(20) NOT NULL,
  `name` VARCHAR(512) NOT NULL,
  `labeller` VARCHAR(256),
  `strength` VARCHAR(50),
  `route` TEXT,
  `approved` BOOLEAN DEFAULT FALSE,
  `ndc_product_code` VARCHAR(20),
  `ema_ma_number` VARCHAR(20),
  FOREIGN KEY (`derived_from`) REFERENCES `drugs`(`id`) ON DELETE CASCADE
);

-- Connections between drugs and products
/*
-- No more required: `dervied_from` in `products` table references `drug_id` in
`darugs` table.
CREATE TABLE IF NOT EXISTS `drug_products` (
  `drug_id` VARCHAR(20) NOT NULL,
  `product_id` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`drug_id`, `product_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`product_id`) REFERENCES `products`(`id`) ON DELETE CASCADE
);
*/

CREATE TABLE IF NOT EXISTS `experimental_properties` (
  -- id: Specially converted from uuid
  `id` BINARY(16) NOT NULL PRIMARY KEY,
  `derived_from` VARCHAR(20) NOT NULL,
  `kind` VARCHAR(50) NOT NULL,
  `value` VARCHAR(256) NOT NULL,
```

```sql
  `unit` VARCHAR(1000) DEFAULT NULL,
  FOREIGN KEY (`derived_from`) REFERENCES `drugs`(`id`) ON DELETE CASCADE
);

-- Connections between drugs and experimental properties
/*
-- No more required: `dervied_from` in `experimental_properties` table references
`drug_id` in `darugs` table.
CREATE TABLE IF NOT EXISTS `drug_experimental_properties` (
  `drug_id` VARCHAR(20) NOT NULL,
  `property_id` BINARY(16) NOT NULL,
  PRIMARY KEY (`drug_id`, `property_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`property_id`) REFERENCES `experimental_properties`(`id`) ON DELETE
CASCADE
);
*/

CREATE TABLE IF NOT EXISTS `targets` (
  `id` VARCHAR(20) NOT NULL PRIMARY KEY,
  `name` VARCHAR(256) NOT NULL,
  `organism` VARCHAR(256)
);

-- Connections between drugs and targets
CREATE TABLE IF NOT EXISTS `drug_targets` (
  `drug_id` VARCHAR(20) NOT NULL,
  `target_id` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`drug_id`, `target_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`target_id`) REFERENCES `targets`(`id`) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS `references` (
  `ref_id` VARCHAR(20) NOT NULL PRIMARY KEY,
  `pubmed_id` VARCHAR(20) NOT NULL,
  `citation` TEXT NOT NULL
);

-- Connections between drugs and references
CREATE TABLE IF NOT EXISTS `drug_references` (
  `drug_id` VARCHAR(20) NOT NULL,
  `ref_id` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`drug_id`, `ref_id`),
  FOREIGN KEY (`drug_id`) REFERENCES `drugs`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`ref_id`) REFERENCES `references`(`ref_id`) ON DELETE CASCADE
);
```

3-insert-from-csv.ipynb

```python
%pip install python-dotenv pymysql
```

```python
CSV_PATH_PREFIX = "../../2/outputs/"
CSV_PATH_DRUGS = CSV_PATH_PREFIX + "Drugs.csv"
CSV_PATH_CATEGORIES = CSV_PATH_PREFIX + "Categories.csv"
CSV_PATH_DRUG_CATEGORY_RELATIONS = CSV_PATH_PREFIX + "DrugCategoryRelations.csv"
CSV_PATH_EXTERNAL_DB_IDS = CSV_PATH_PREFIX + "ExternalDatabaseIdentifiers.csv"
```

```
CSV_PATH_PRODUCTS = CSV_PATH_PREFIX + "Products.csv"
CSV_PATH_EXPERIMENTALS = CSV_PATH_PREFIX + "ExperimentalProperties.csv"
CSV_PATH_TARGETS = CSV_PATH_PREFIX + "Targets.csv"
CSV_PATH_DRUG_TARGET_RELATIONS = CSV_PATH_PREFIX + "DrugTargetRelations.csv"
CSV_PATH_REFERENCES = CSV_PATH_PREFIX + "References.csv"
CSV_PATH_DRUG_REFERENCE_RELATIONS = CSV_PATH_PREFIX + "DrugReferenceRelations.csv"

SQL_INSERT_TO_DRUGS = "INSERT INTO drugs (id, name, description, cas_number, state,
unii) VALUES(%s, %s, %s, %s, %s, %s);"
SQL_INSERT_TO_CATEGORIES = "INSERT INTO categories (id, name) VALUES(%s, %s);"
SQL_INSERT_TO_DRUG_CATEGORY_RELATIONS = "INSERT INTO drug_categories (drug_id,
category_id) VALUES(%s, %s);"
SQL_INSERT_TO_EXTERNAL_DB_IDS = "INSERT INTO external_db_identifiers (id, external_id,
derived_from, resource) VALUES(%s, %s, %s, %s);"
SQL_INSERT_TO_PRODUCTS = "INSERT INTO products (id, derived_from, name, labeller,
strength, route, approved, ndc_product_code, ema_ma_number) VALUES(%s, %s, %s, %s, %s,
%s, %s, %s, %s);"
SQL_INSERT_TO_EXPERIMENTALS = "INSERT INTO experimental_properties (id, derived_from,
kind, value, unit) VALUES(%s, %s, %s, %s, %s);"
SQL_INSERT_TO_TARGETS = "INSERT INTO targets (id, name, organism) VALUES(%s, %s, %s);"
SQL_INSERT_TO_DRUG_TARGET_RELATIONS = "INSERT INTO drug_targets (drug_id, target_id)
VALUES(%s, %s);"
SQL_INSERT_TO_REFERENCES = "INSERT INTO `references` (ref_id, pubmed_id, citation)
VALUES(%s, %s, %s);"
SQL_INSERT_TO_DRUG_REFERENCE_RELATIONS = "INSERT INTO drug_references (drug_id,
ref_id) VALUES(%s, %s);"
```

```
import pymysql
from dotenv import dotenv_values
import csv
import uuid

config = dotenv_values(".env")
uuidgen = uuid.uuid1

def uuid_optm(uuid: str) -> bytes:
    field = uuid.split('-')
    hex = field[2] + field[1] + field[0] +field[3] + field[4]
    binary = bytes.fromhex(hex)
    return binary
```

```
db = pymysql.connect(
    host=config["host"],
    port=int(config["port"]),
    user=config["user"],
    passwd=config["passwd"],
    db=config["db"],
    charset=config["charset"]
)

curs = db.cursor()
```

```
with open(CSV_PATH_DRUGS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_DRUGS, line)
```

```
with open(CSV_PATH_CATEGORIES, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_CATEGORIES, line)
```

```
with open(CSV_PATH_DRUG_CATEGORY_RELATIONS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_DRUG_CATEGORY_RELATIONS, line)
```

```
with open(CSV_PATH_EXTERNAL_DB_IDS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        _id = uuid_optm(str(uuidgen()))
        line = [_id, *line]
        curs.execute(SQL_INSERT_TO_EXTERNAL_DB_IDS, line)
```

```
with open(CSV_PATH_PRODUCTS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        _id = uuid_optm(str(uuidgen()))
        line = [_id, *line]
        line[6] = bool(line[6])
        curs.execute(SQL_INSERT_TO_PRODUCTS, line)
```

```
with open(CSV_PATH_EXPERIMENTALS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        _id = uuid_optm(str(uuidgen()))
        line = [_id, *line]
        curs.execute(SQL_INSERT_TO_EXPERIMENTALS, line)
```

```
with open(CSV_PATH_TARGETS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_TARGETS, line)
```

```
with open(CSV_PATH_DRUG_TARGET_RELATIONS, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_DRUG_TARGET_RELATIONS, line)
```

```
with open(CSV_PATH_REFERENCES, 'r') as f:
    rdr = csv.reader(f)
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_REFERENCES, line)
```

```
with open(CSV_PATH_DRUG_REFERENCE_RELATIONS, 'r') as f:
    rdr = csv.reader(f)
```

```
    rdr.__next__()
    for line in rdr:
        curs.execute(SQL_INSERT_TO_DRUG_REFERENCE_RELATIONS, line)
```

```
db.commit()
```

4. 각 테이블에 index 설정 코드 및 설정결과

4-init-index.sql

```sql
USE `drugbank_db`;

SHOW INDEX FROM `drugs`;
SHOW INDEX FROM `categories`;
SHOW INDEX FROM `drug_categories`;
SHOW INDEX FROM `external_db_identifiers`;
SHOW INDEX FROM `products`;
SHOW INDEX FROM `experimental_properties`;
SHOW INDEX FROM `targets`;
SHOW INDEX FROM `drug_targets`;
SHOW INDEX FROM `references`;
SHOW INDEX FROM `drug_references`;

CREATE INDEX `idx_drug_name` ON `drugs` (`name`);
CREATE INDEX `idx_drug_external_id` ON `external_db_identifiers` (`external_id`);
CREATE INDEX `idx_product_name` ON `products` (`name`);
CREATE INDEX `idx_experimental_property_kind` ON `experimental_properties` (`kind`);
CREATE INDEX `idx_target_name` ON `targets` (`name`);
CREATE INDEX `idx_drug_reference` ON `drug_references` (`drug_id`, `ref_id`);
CREATE INDEX `idx_drug_target` ON `drug_targets` (`drug_id`, `target_id`);
CREATE INDEX `idx_drug_category` ON `drug_categories` (`drug_id`, `category_id`);
```

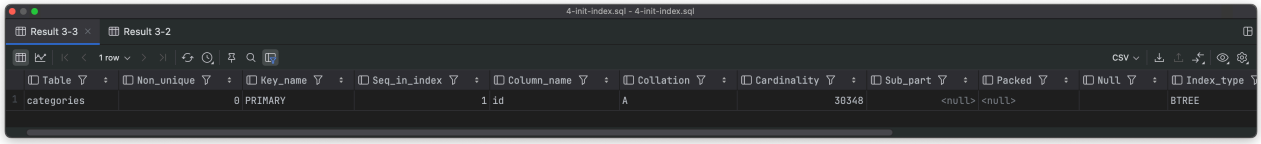


	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	categories	0	PRIMARY	1	id	A	30348	<null>	<null>		BTREE

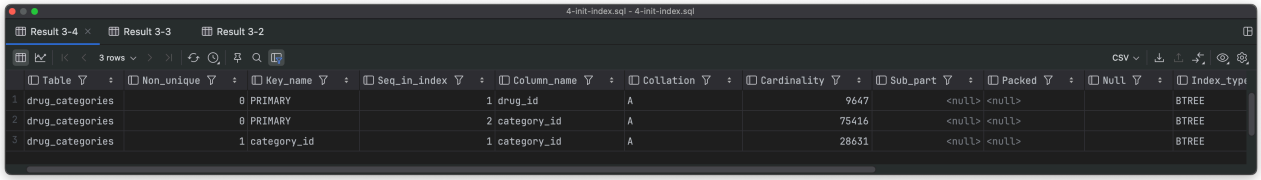


	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	drug_categories	0	PRIMARY	1	drug_id	A	9647	<null>	<null>		BTREE
2	drug_categories	0	PRIMARY	2	category_id	A	75416	<null>	<null>		BTREE
3	drug_categories	1	category_id	1	category_id	A	28631	<null>	<null>		BTREE

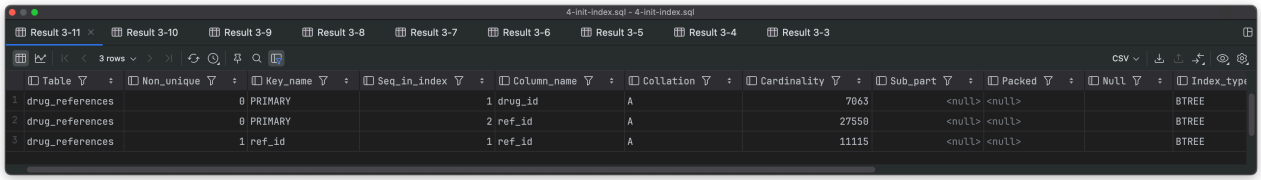


	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	drug_references	0	PRIMARY	1	drug_id	A	7063	<null>	<null>		BTREE
2	drug_references	0	PRIMARY	2	ref_id	A	27550	<null>	<null>		BTREE
3	drug_references	1	ref_id	1	ref_id	A	11115	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Result 3-9 | Result 3-8 | Result 3-7 | Result 3-6 | Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

3 rows

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	drug_targets	0	PRIMARY	1	drug_id	A	7327	<null>	<null>		BTREE
2	drug_targets	0	PRIMARY	2	target_id	A	18675	<null>	<null>		BTREE
3	drug_targets	1	target_id	1	target_id	A	4387	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Result 3-2

1 row

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	drugs	0	PRIMARY	1	id	A	15197	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Result 3-7 | Result 3-6 | Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

2 rows

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	I
1	experimental_properties	0	PRIMARY	1	id	A	7271	<null>	<null>		BTRE
2	experimental_properties	1	derived_from	1	derived_from	A	2645	<null>	<null>		BTRE

4-init-index.sql - 4-init-index.sql

Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

2 rows

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	I
1	external_db_identifiers	0	PRIMARY	1	id	A	83384	<null>	<null>		BTRE
2	external_db_identifiers	1	derived_from	1	derived_from	A	14027	<null>	<null>		BTRE

4-init-index.sql - 4-init-index.sql

Result 3-6 | Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

2 rows

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	products	0	PRIMARY	1	id	A	286989	<null>	<null>		BTREE
2	products	1	derived_from	1	derived_from	A	3045	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Result 3-10 | Result 3-9 | Result 3-8 | Result 3-7 | Result 3-6 | Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

1 row

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	references	0	PRIMARY	1	ref_id	A	12455	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Result 3-8 | Result 3-7 | Result 3-6 | Result 3-5 | Result 3-4 | Result 3-3 | Result 3-2

1 row

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
1	targets	0	PRIMARY	1	id	A	5024	<null>	<null>		BTREE

4-init-index.sql - 4-init-index.sql

Output

```
drugbank_db> CREATE INDEX `idx_drug_name` ON `drugs` (`name`)
[2025-05-30 20:31:40] completed in 295 ms
drugbank_db> CREATE INDEX `idx_drug_external_id` ON `external_db_identifiers` (`external_id`)
[2025-05-30 20:31:41] completed in 208 ms
drugbank_db> CREATE INDEX `idx_product_name` ON `products` (`name`)
[2025-05-30 20:31:41] completed in 560 ms
drugbank_db> CREATE INDEX `idx_experimental_property_kind` ON `experimental_properties` (`kind`)
[2025-05-30 20:31:41] completed in 31 ms
drugbank_db> CREATE INDEX `idx_target_name` ON `targets` (`name`)
[2025-05-30 20:31:41] completed in 49 ms
drugbank_db> CREATE INDEX `idx_drug_reference` ON `drug_references` (`drug_id`, `reference_id`)
[2025-05-30 20:31:41] [42000][1072] Key column 'reference_id' doesn't exist in table
drugbank_db> CREATE INDEX `idx_drug_reference` ON `drug_references` (`drug_id`, `ref_id`)
[2025-05-30 20:31:48] completed in 96 ms
drugbank_db> CREATE INDEX `idx_drug_target` ON `drug_targets` (`drug_id`, `target_id`)
[2025-05-30 20:31:50] completed in 78 ms
drugbank_db> CREATE INDEX `idx_drug_category` ON `drug_categories` (`drug_id`, `category_id`)
[2025-05-30 20:31:56] completed in 203 ms
```

5. 각 테이블 별로 포함된 tuple 예시 출력

5-select-random.sql

```sql
USE `drugbank_db`;

SELECT * FROM `drugs` ORDER BY RAND() LIMIT 3;
SELECT * FROM `categories` ORDER BY RAND() LIMIT 3;
SELECT * FROM `drug_categories` ORDER BY RAND() LIMIT 3;
SELECT * FROM `external_db_identifiers` ORDER BY RAND() LIMIT 3;
SELECT * FROM `products` ORDER BY RAND() LIMIT 3;
SELECT * FROM `experimental_properties` ORDER BY RAND() LIMIT 3;
SELECT * FROM `targets` ORDER BY RAND() LIMIT 3;
SELECT * FROM `drug_targets` ORDER BY RAND() LIMIT 3;
SELECT * FROM `references` ORDER BY RAND() LIMIT 3;
SELECT * FROM `drug_references` ORDER BY RAND() LIMIT 3;
```

5-select-random.sql - 5-select-random.sql

drugbank_db.categories | drugbank_db.drugs

3 rows

	id	name
1	A001442	Cytochrome P-450 Substrates
2	A019924	Agents for Treatment of Hemorrhoids and Anal Fissures for Topical Use
3	A015244	Cytochrome P-450 CYP2C9 Inhibitors (strength unknown)

5-select-random.sql - 5-select-random.sql

drugbank_db.drug_categories | drugbank_db.categories | drugbank_db.drugs

3 rows

	drug_id	category_id
1	DB04869	D000072471
2	DB00522	D020313
3	DB01320	A012511

5-select-random.sql - 5-select-random.sql

drugbank_db.drug_references | drugbank_db.references | drugbank_db.drug_targets | drugbank_db.targets | drugbank_db.experimental_properties | drugba

3 rows

	drug_id	ref_id
1	DB00197	A20166
2	DB01221	A18290
3	DB03042	A1715

5-select-random.sql - 5-select-random.sql

drugbank_db.drug_targets | drugbank_db.targets | drugbank_db.experimental_properties | drugbank_db.products | drugbank_db.external_db_identifiers | dr

3 rows

	drug_id	target_id
1	DB14533	BE0005833
2	DB03573	BE0001504
3	DB03507	BE0001139

5-select-random.sql - 5-select-random.sql

drugbank_db.drugs

3 rows

	id	name	description	cas_number	state	unii
1	DB16672	Epcoritamab	Epcoritamab is under investigation in cli…	2134641-34-0		D6OMY2L0WA
2	DB03941	p-Heptylphenol		1987-50-4	solid	I743Z35DVB
3	DB14733	Ceftobiprole medocaril	Ceftobiprole medocaril is a [ceftobiprole…	376653-43-9		YXV28V1B07

5-select-random.sql - 5-select-random.sql

drugbank_db.experimental_properties | drugbank_db.products | drugbank_db.external_db_identifiers | drugbank_db.drug_categories | drugbank_db.categories

3 rows

	id (UUID with time-low and time…	derived_from	kind	value	unit
1	8e662192-3d37-11f0-8f74-e2c01f351b76	DB00446	logS	-2.11	ADME Research, USCD
2	8e79d1d8-3d37-11f0-8f74-e2c01f351b76	DB01183	logP	2.09	HANSCH,C ET AL. (1995)
3	8e653138-3d37-11f0-8f74-e2c01f351b76	DB00417	pKa	2.79	SANGSTER (1994)

5-select-random.sql - 5-select-random.sql

drugbank_db.external_db_identifiers | drugbank_db.drug_categories | drugbank_db.categories | drugbank_db.drugs

3 rows

	id (UUID with time-low and time…	external_id	derived_from	resource
1	7e7b2cfa-3d37-11f0-8f74-e2c01f351b76	LYA	DB00642	PDB
2	7ef30716-3d37-11f0-8f74-e2c01f351b76	PA153906323	DB01954	PharmGKB
3	7f9c2378-3d37-11f0-8f74-e2c01f351b76	347910076	DB05315	PubChem Substance

5-select-random.sql - 5-select-random.sql

drugbank_db.products | drugbank_db.external_db_identifiers | drugbank_db.drug_categories | drugbank_db.categories | drugbank_db.drugs

3 rows

	id (UUID with time-low and time…	derived_from	name	labeller	strength	route
1	85e6f852-3d37-11f0-8f74-e2c01f351b76	DB00709	Lamivudine	Major Pharmaceuticals	150 mg/1	Oral
2	87f3e8ee-3d37-11f0-8f74-e2c01f351b76	DB00912	Repaglinide	Pro Doc Limitee	2 mg	Oral
3	873d890a-3d37-11f0-8f74-e2c01f351b76	DB00898	GNC Antimicrobial Hand Wipes	GNC Holdings LLC	66.5 mL/100mL	Topical

5-select-random.sql - 5-select-random.sql

drugbank_db.references | drugbank_db.drug_targets | drugbank_db.targets | drugbank_db.experimental_properties | drugbank_db.products | drugbank_db.

3 rows | Tx: Auto | DDL | CSV

	ref_id	pubmed_id	citation
1	A10360	15283690	Jiang W, Miyamoto T, Kakizawa T, Sakuma T, N
2	A13438	16426349	Ustunsoy H, Sivrikoz MC, Tarakcioglu M, B…
3	A12287	16574083	Song Y, Wei EQ, Zhang WP, Ge QF, Liu JR, …

5-select-random.sql - 5-select-random.sql

drugbank_db.targets | drugbank_db.experimental_properties | drugbank_db.products | drugbank_db.external_db_identifiers | drugbank_db.drug_categories

3 rows | Tx: Auto | DDL | CSV

	id	name	organism
1	BE0001966	6-phosphogluconolactonase	Thermotoga maritima (strain ATCC 43589 / …
2	BE0002407	Phosphatidylinositol 4,5-bisphosphate 3-k…	Humans
3	BE0001212	Beta-lactamase type II	Bacteroides fragilis